

Partie 1

# Rappels

# Organisation du cours

## Volume horaire :

- 10 séances de cours d'1h30 (15h au total)
- 11 séances de TP de 4h (44h au total)

## Évaluation :

- un examen terminal en janvier (12 points)
- contrôle continu, dont partiel (4 points)
- travaux pratiques (4 points)

# Objectifs du cours

- Perfectionner ses compétences en Java.
- Apprendre à concevoir avec des objets :
  - décomposer un projet en classes,
  - choisir les propriétés et méthodes des classes,
  - concevoir les interactions entre les classes.
- Apprendre à programmer proprement, à utiliser des outils de développement professionnels.

# Déroulement des cours

Fonctionnement en classe inversée :

- on **prépare le cours** en lisant les transparents avant le cours, et on répond au **test d'avant cours**,
- en cours, on discute des points difficiles dans le cours, sous la forme de quizz ou d'exercices,
- en TP, vous commencerez par une série d'exercices indépendants (entre 1h et 2h), puis vous travaillerez sur un problème plus long.

**Ne fonctionne que si vous jouez le jeu !**

---

# Structures des programmes

---

# Qu'est-ce que programmer ?

- 1 Analyser les besoins
- 2 Spécifier les comportements du programme
- 3 Choisir et éventuellement concevoir les solutions techniques
- 4 Implémenter le programme
- 5 Vérifier que le programme a le comportement spécifié (tester)
- 6 Déployer le programme dans son environnement
- 7 Maintenir le programme (corriger les bugs, ajouter des fonctionnalités)

# Programmation orienté objet

- Décomposer le programme en unité (les objets),
- Chaque unité a une unique responsabilité,
- Les unités sont indépendantes,
- Les unités communiquent entre elles par messages (appels de méthodes)

# Organisation d'un programme Java

- tâches réalisées par des objets (objet = unité agissante),
- les objets ayant les mêmes comportements sont définis par une **classe** (classe = **comment** l'objet agit),
- les interfaces permettent d'abstraire les comportements des objets (interface = **quoi** fait l'objet),
- la classe `Main` contient une méthode `main` qui crée les premiers objets et initie la réalisation des tâches à accomplir par le programme.

# Un objet

Un objet est défini par :

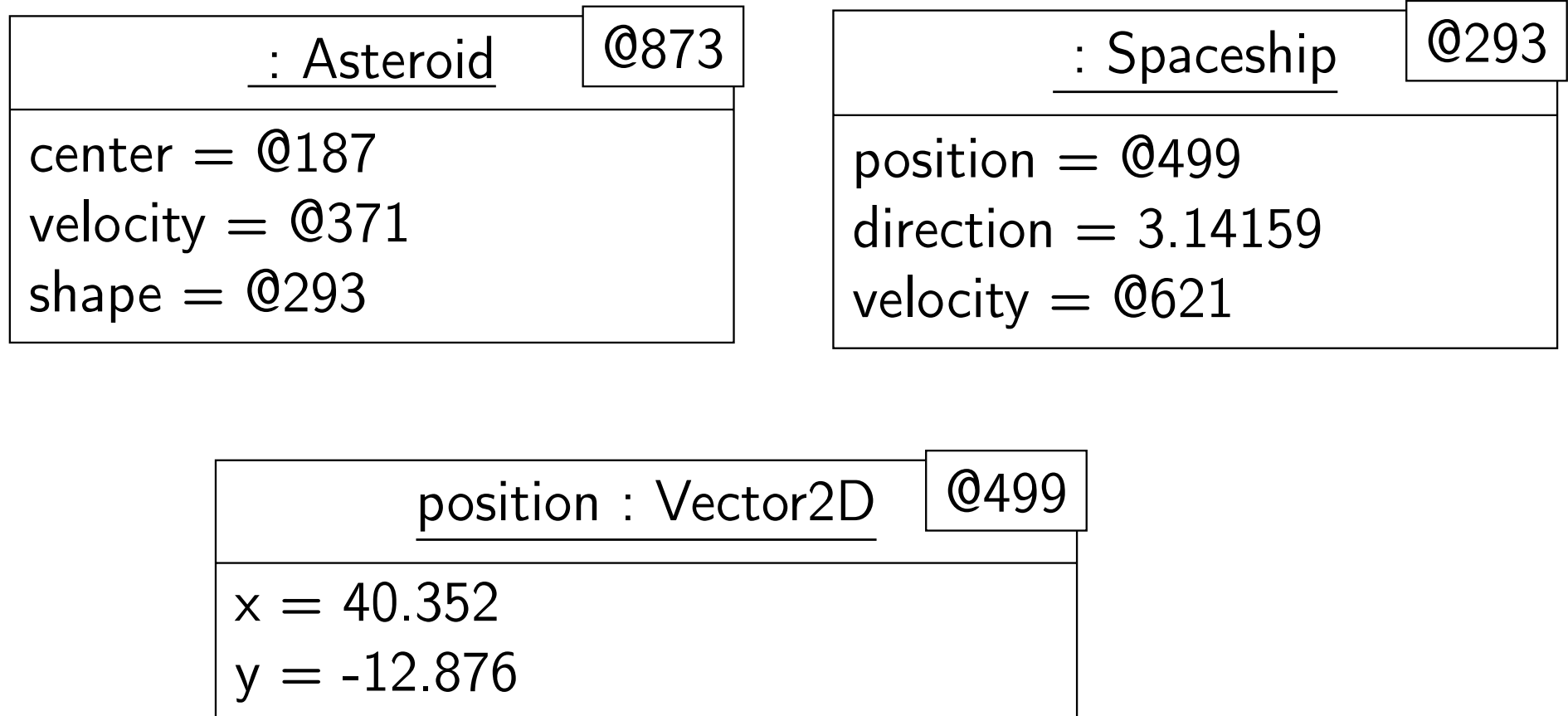
- ses méthodes (ce qu'il fait),
- ses propriétés (des valeurs internes à l'objet, qui lui servent à accomplir sa tâche).

Les propriétés et méthodes peuvent être :

- publique (accessible à tous),
- ou privée (seulement à lui-même).

# Diagrammes d'objets

Les diagrammes d'objets permettent de représenter l'état du programme à un instant de son exécution.



# Une classe

Une classe définit :

- une façon de construire et initialiser un objet,
- le comportement des méthodes de cet objet,
- les propriétés de cet objet.

**Remarques :**

- Les classes ne sont pas nécessaires pour définir des objets, mais c'est la façon standard.
- on peut construire autant d'objets que voulus d'une même classe.

# Une interface

Une interface définit :

- une liste de méthodes pouvant être réalisés par des objets.

Remarques :

- tout objet peut implémenter zéro, une ou plusieurs interfaces,
- les interfaces sont des types : elles peuvent être utilisés pour préciser la nature d'un paramètre d'une méthode, *etc.*

# Relations objet vs classe vs interface

- un objet **instancie** une classe (et une seule),
  - une classe peut **implémenter** une ou plusieurs interfaces,
  - une interface peut **étendre** une autre interface
- 
- une interface peut être implémentée par plusieurs classes,
  - une classe peut être instanciée en plusieurs objets,
  - une classe **définie** des propriétés et méthodes,
  - un objet **possède** des propriétés et méthodes.

# Un projet en Java

Un projet est constitué de :

- *packages*, qui regroupent plusieurs fichiers dans un même *espace de nom*, constitués de...
- *fichiers*, correspondant chacun à une classe ou une interface, et comprenant chacun...
- d'une *déclaration de classe ou d'interface*.

```
public class MyClassName { ... }
```

```
public interface MyInterfaceName { ... }
```

# Une classe en Java

Une classe contient (dans sa déclaration) :

- des **déclarations de propriétés** (avec éventuellement une initialisation),
- des **déclarations de méthodes et de constructeurs**, chacune contenant un bloc d'instructions.

Une interface contient :

- des **déclarations de méthodes** (sans implémentation).

# Déclaration de propriétés

Exemples sans initialisation (initialisés par défaut) :

```
private int count;  
private final Color backgroundColor;  
private final String name;
```

Exemples avec initialisation :

```
private int count = 0;  
private final Color backColor = new Color(0.5,0.2,0);  
private final String name = "John Doe";
```

# Déclaration d'une méthode

visibilité + type de retour + nom + (paramètres) + {  
    bloc d'instructions  
}

Exemples :

```
public int getCount() { ... }  
  
public void setCount(int newCount) { ... }  
  
public void launchMissile(GPSCoordinate coord) { ... }  
  
private Vector3D getAcceleration() { ... }
```

---

# Écriture des méthodes

---

# Corps d'une méthode

Le corps d'une méthode ou d'une classe est composé :

- d'**instructions**, terminées par ;,
- de **structures de contrôle**, permettant de décider quelles instructions exécuter.

## Remarques :

- Sans structure de contrôles, les instructions sont exécutées une par une, dans le sens de lecture.
- On écrit une instruction ou une structure de contrôle par ligne.
- On utilise l'**indentation** pour mettre en valeur les structures de contrôle.

# Composants d'une méthode

Il nous faut donc savoir :

- utiliser les 5 (principales) instructions disponibles,
- écrire des expressions servant de paramètres pour ces expressions,
- contrôler l'exécution des instructions selon des conditions (exprimées par des expressions booléennes) avec les structures conditionnelles et les boucles.

Nous détaillons tous ces concepts dans les prochains transparents.

# Les instructions : création d'une variable

Une **variable** est un espace mémoire nommé pour stocker une valeur **momentanément**.

**Notion de portée** : La variable existe

- depuis le moment où l'instruction de création est exécutée,
- jusqu'à la sortie du bloc (`{ ... }`) où elle est définie.

La variable possède un **type** (primitif, classe ou interface), et ne peut contenir que des valeurs de ce type.

```
int count;  
Vector2D position;
```

# Les instructions : l'affectation

L'**affectation** est l'opération consistant à mettre une valeur dans un espace mémoire défini par la ***l-value*** (left-value, le terme de gauche dans l'affectation), généralement une variable.

L'affectation est soumise aux contraintes de types : la valeur stockée doit être d'un type compatible avec celui déclaré.

```
count = 12;  
count = count + 4;  
position = new Vector(3, -2.5);
```

La ***r-value*** (le terme de droite) doit être une expression.

# Les left-values

La left-value d'une affectation peut être :

- une variable de la méthode, dont la portée englobe cette instruction,
- un paramètre de la méthode,
- une variable d'objet = une propriété de l'objet,
  - de l'objet exécutant cette méthode : `this.someVar`,
  - d'un autre objet : `someObject.someVar`,
- une variable de classe = propriété `static`, avec `SomeClass.someVar`,
- la cellule d'un tableau : `someArray[i]`.

# Les instructions : la construction

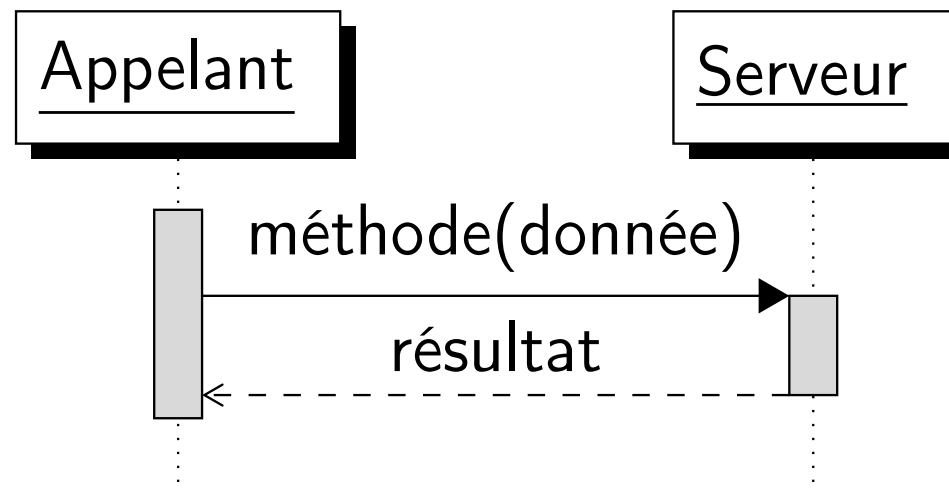
La création d'un nouvel objet se fait avec le mot-clé `new`, qui déclenche :

- 1 l'allocation d'un espace mémoire pour contenir le nouvel objet,
- 2 l'initialisation des propriétés de cet objet,
- 3 l'appel au constructeur de l'objet,
- 4 l'adresse mémoire de l'objet est retournée par l'instruction.

```
new Vector(3,-2.5);  
new Spaceship();  
new ArrayList<Integer>();
```

# Les instructions : l'appel de méthode

L'**appel de méthode** permet d'envoyer un message à un autre objet, déclenchant les instructions de la méthode de cet objet. On utilise les diagrammes de séquences pour représenter les appels de méthodes. Ce sont des **diagrammes temporels**, montrant les interactions entre les objets.



```
server.doSomething(someData);
```

# Les instructions : `return`

La méthode peut retourner une valeur lorsqu'elle a terminé son travail, en utilisant le mot-clé `return`.

Lors de l'évaluation de l'instruction `return`, l'argument du `return` (le résultat) est **d'abord** évalué (s'il existe), **ensuite** l'exécution de la méthode est interrompue. Le résultat est retourné au niveau de l'instruction ayant fait l'appel de méthode.

```
return;  
return 42;  
return new Vector(-5,2.333);
```

# Les instructions : récapitulatif

Les instructions sont :

- la création d'une variable,
- l'affectation,
- la construction d'un objet,
- l'appel d'une méthode (de type de retour `void`),
- la terminaison et le retour de la méthode.

La création d'une variable peut-être combinée avec une affectation.

```
String name = "John Doe";  
Car blueCar = new Car(Color.BLUE);
```

# Les expressions

Les valeurs sont définies par des **expressions** :

- littéral arithmétique (0, 3.14, ...) ou booléen (**true**, **false**), chaînes de caractères (entre "..."),
- variables et propriétés,
- appels de méthode (ayant un type de retour autre que **void**),
- instruction **new**,
- combinaison d'expressions par des opérateurs (+, \*, ...),  
...

Les expressions apparaissent :

- en membre droit d'une affectation (*r-value*),
- en paramètre d'un appel de méthode, d'un constructeur,
- en argument d'une instruction **return**,
- en condition d'une structure de contrôle.

# Les variables d'une expression

Quelles variables sont utilisables dans une expression ?

- Les variables de la méthode, si cette expression est dans leur portée : `someVar`.
- Les paramètres de la méthode : `someParam`.
- Les variables de l'objet (= propriétés) exécutant la méthode : `this.someVar`.
- Les variables d'objet (= propriétés) d'un autre objet, du moment qu'elles sont publiques : `someObject.someVar`.
- Les variables de classe (= propriétés statiques) de la classe, `someVar`, ou d'une autre classe si elle sont publiques : `SomeClass.someVar`.

# Les structures de contrôles : if

La **structure conditionnelle** permet d'exécuter un bloc d'instructions que si une condition est satisfaite.

```
if (people.isGoodGuy()) {  
    Sout.println("Greetings, " + people.getName() + "!");  
}  
  
if (people.isGoodGuy()) {  
    Sout.println("Greetings, " + people.getName() + "!");  
} else {  
    Sout.println("You are not welcome here, "  
                + people.getName() + ".");  
}
```

# Organigramme de programmation

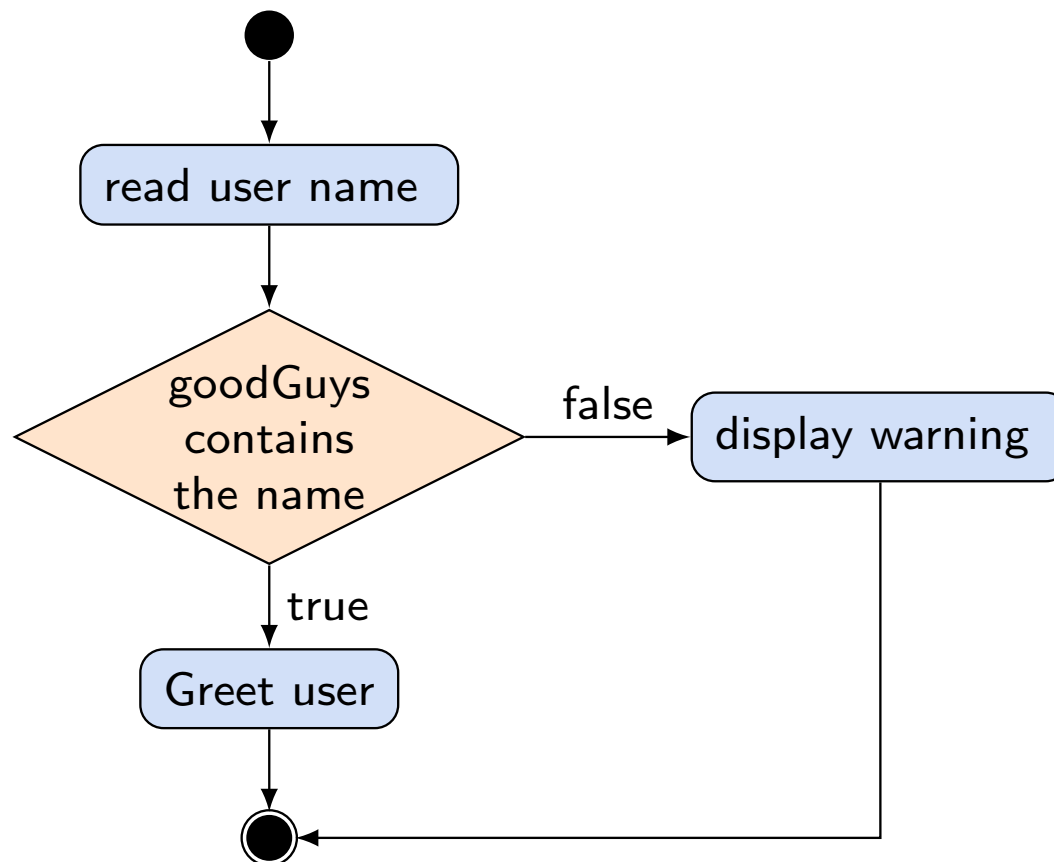
Pour représenter l'ordre d'exécution des instructions, on utilise des diagrammes appelés **organigramme de programmation** (*flowchart* en anglais).

- Chaque étape élémentaire est représenté par un bloc rectangulaire,
- L'ordre entre les étapes est donné en suivant les flèches,
- le disque noir indique le point de départ, les disques noirs entourées indique un point final.
- les bloc en losange indique les choix, régies par des conditions.

(version simplifiée des diagrammes d'activités en UML).

# Exemple

```
String name = input.readLine();  
if (goodGuys.contains(name)) {  
    Sout.println("Greetings " + name + "!");  
} else {  
    Sout.println("You are not welcome here.");  
}
```



# La répétition indéfinie : `while`

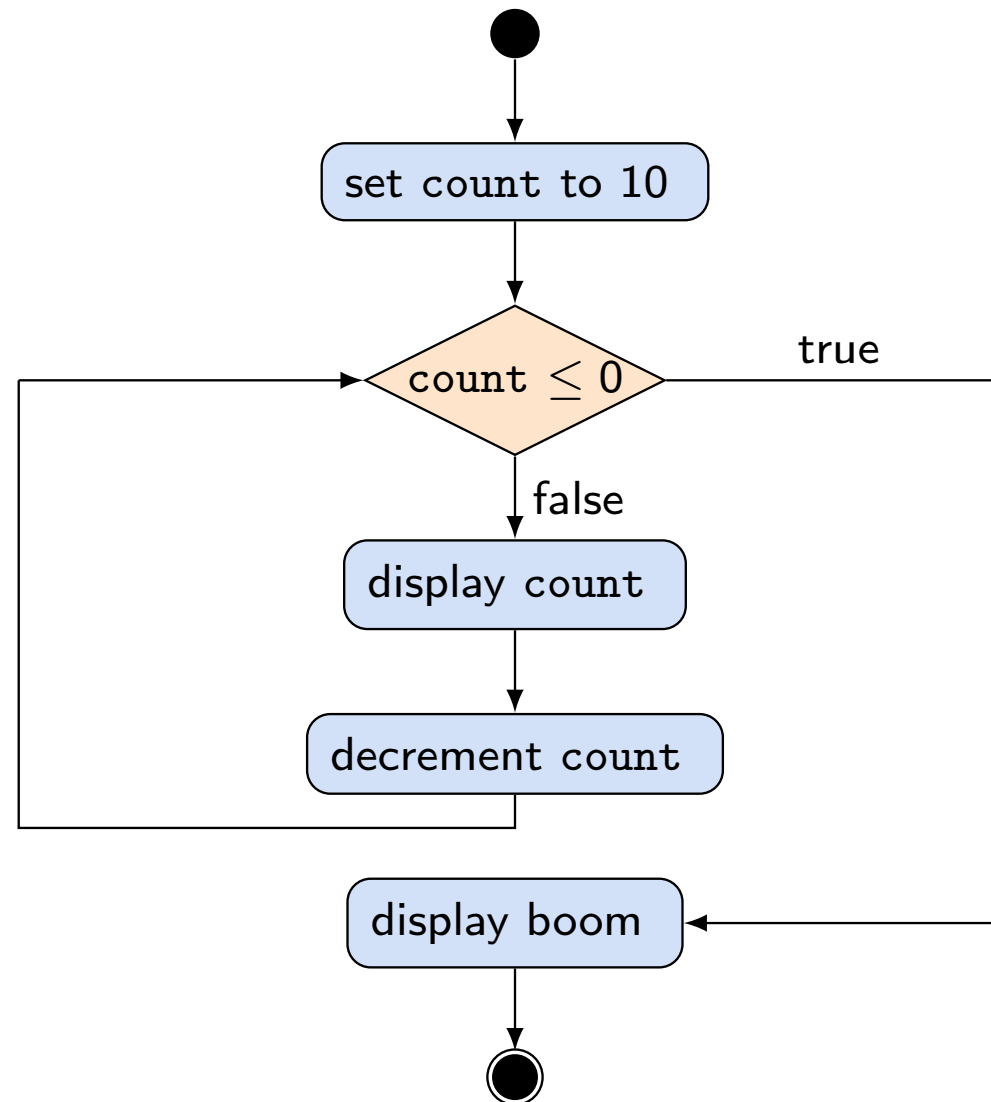
La boucle `while` permet de répéter les mêmes instructions un nombre quelconque de fois. On quitte la boucle avec l'instruction `break` ou l'instruction `return`.

```
// Le compte à rebours  
int count = 10;  
while (true) {  
    if (count <= 0) break;  
    System.out.println(count + "...");  
    count = count - 1;  
}  
System.out.println("BOUM!!!");
```

Note : on peut avoir plusieurs instructions `break` dans la même boucle.

# Organigramme : `while (true)`

Ce qui donne :



# La répétition indéfinie : `while` (II)

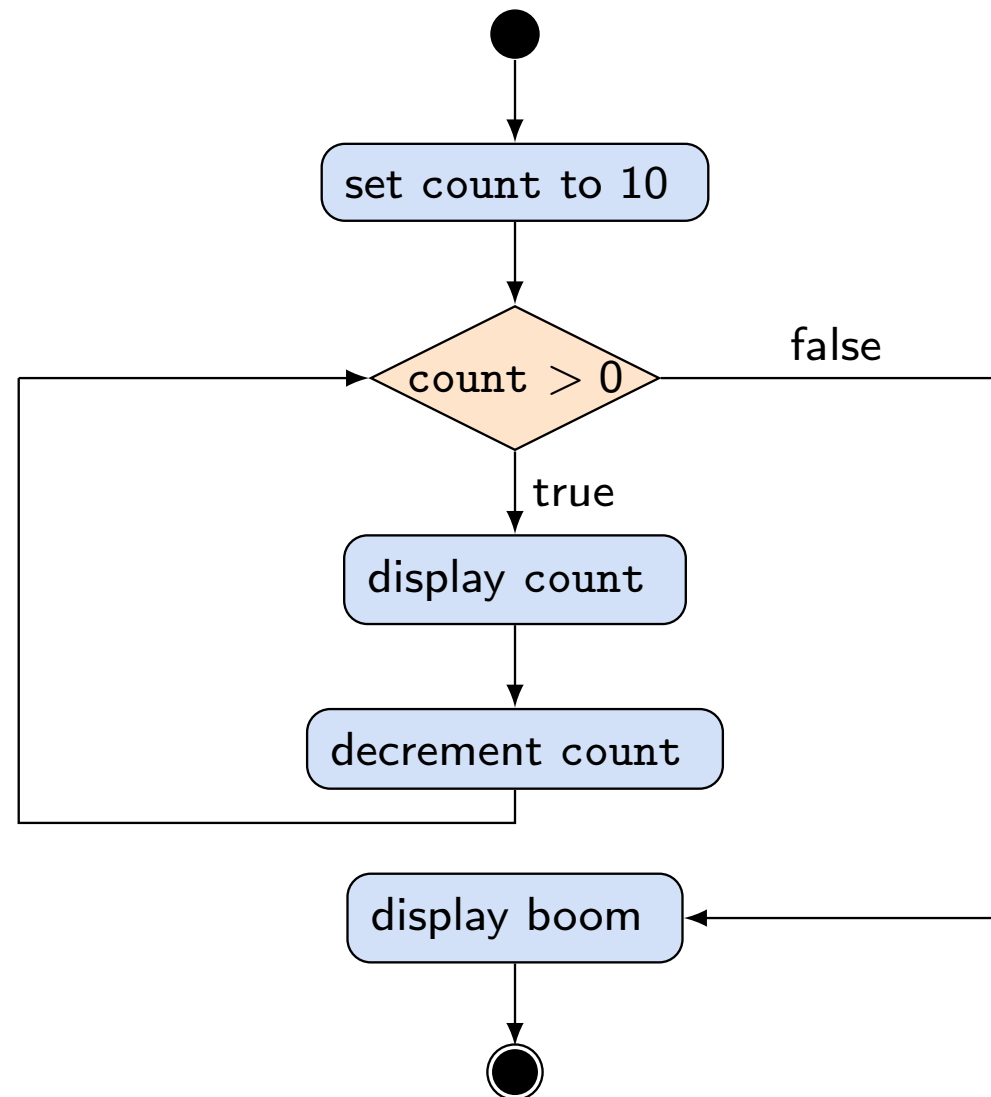
Lorsque le test de sortie est au début de la boucle `while`, on le place (inversé) directement dans la **condition d'arrêt** du `while` :

```
int count = 10;
while (count > 0) {
    System.out.println(count + "...");
    count = count - 1;
}
System.out.println("BOUM!!!");
```

Attention : le test n'est effectué que juste avant d'effectuer la première instruction à chaque itération !

# Organigramme : `while`

Ce qui donne (notez la subtile différence) :



# La répétition indéfinie : `while` (III)

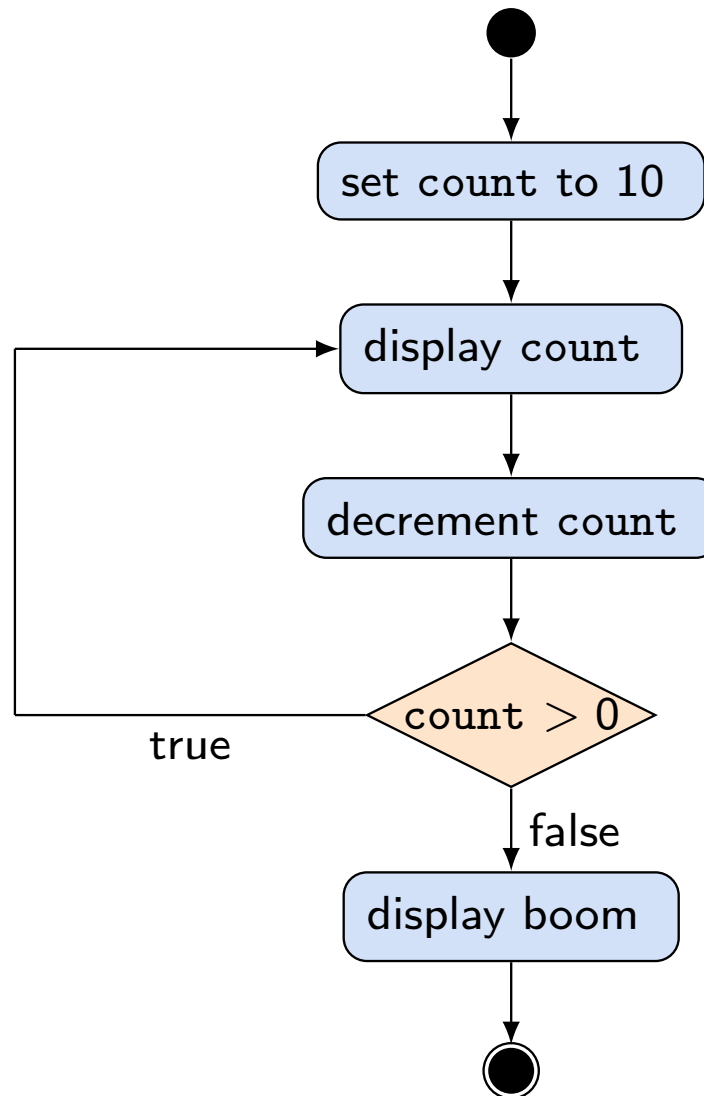
Lorsque le test de sortie est à la fin de la boucle `while`, on utilise la construction `do ... while` :

```
int count = 10;
do {
    System.out.println(count + "...");
    count = count - 1;
} while (count > 0);
System.out.println("BOUM!!!");
```

Attention : le test n'est effectué que juste après d'effectuer la dernière instruction de chaque itération !

# Organigramme : do while

Ce qui donne :



# Comparaison de ces 3 solutions

## Exercice

Que se passe-t-il dans chaque cas si on initialise le compte à rebours à 0 ou -1 ?

En déduire que la dernière version est moins bonne que les autres.

# La répétition bornée : `for`

Dans certains cas, il est plus direct d'utiliser une boucle `for`, qui isole :

- l'initialisation de la boucle,
- la condition d'arrêt,
- l'instruction de progression.

```
for (int count = 10; count > 0; count = count - 1) {  
    System.out.println(count + "...");  
}  
System.out.println("BOUM!!!");
```

On peut utiliser `break` et `return` dans une boucle `for` aussi.

# La répétition bornée : `for`

```
for (initialisation;  
    condition;  
    incrément)  
{  
    ...  
}
```

=

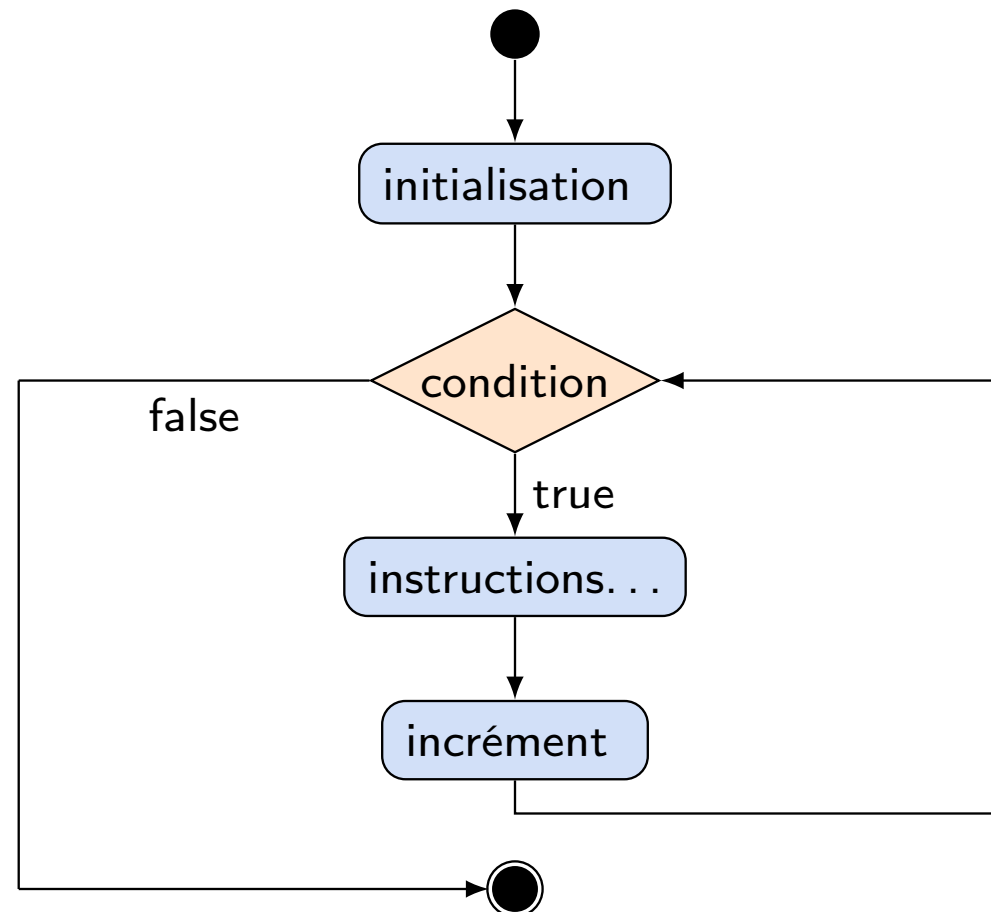
```
initialisation;  
while (condition) {  
    ...  
    increment;  
}
```

Usages typiques :

- exécuter des instructions paramétrées par un indice entier `index`, pour toutes les valeurs de `index` entre 0 et  $n$  (ou un intervalle d'entiers arbitraires),
- exécuter des instructions pour toutes les valeurs d'un tableau (*cf.* prochain cours).

# Organigramme : for

```
for (initialisation; condition; incrément) {  
    instructions;  
}
```



# L'itération : for

Pour les objets qui sont des **collections**, implémentant l'interface `Iterable<Elt>`, on peut utiliser la boucle **for** ainsi :

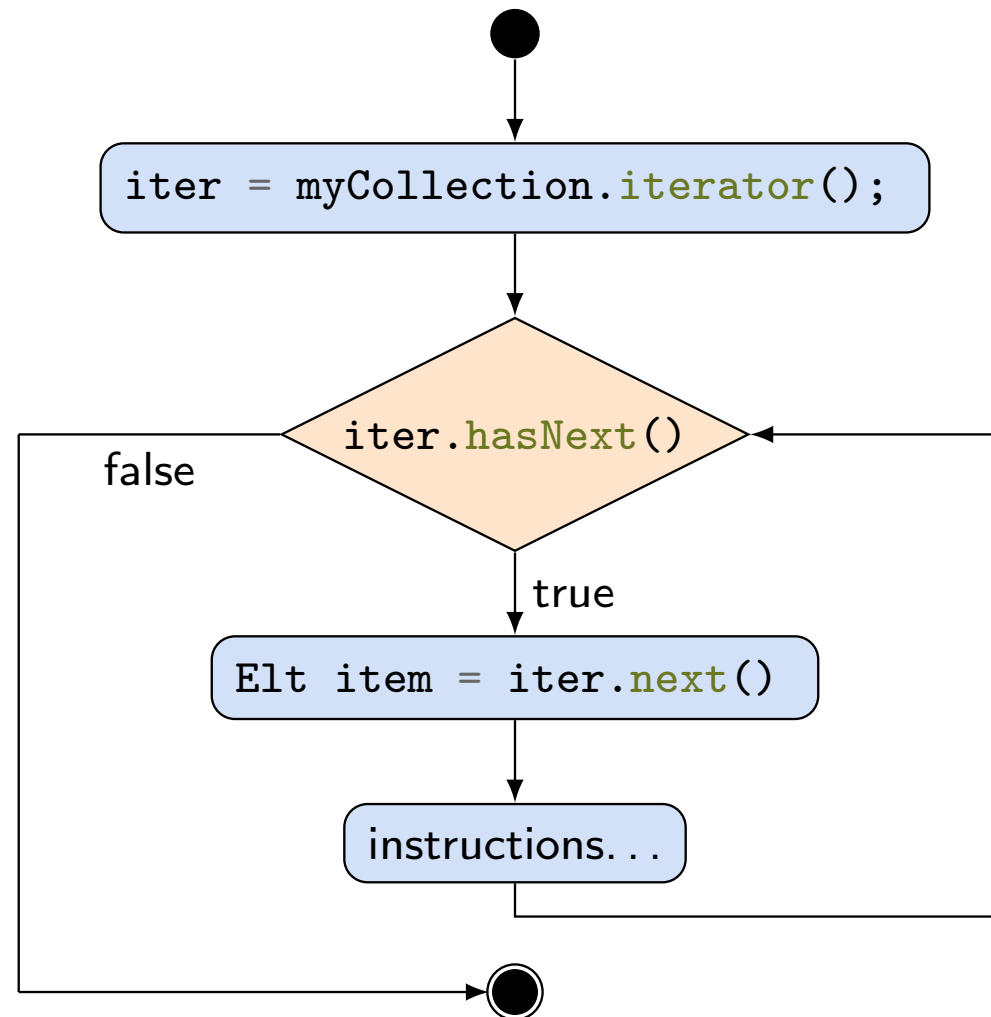
```
for (Elt item : myCollection) {  
    item.doSomething();  
    // ...  
}
```

Abbrévation pour :

```
Iterator<Elt> iter = myCollection.iterator();  
for ( ; iter.hasNext(); ) {  
    Elt item = iter.next();  
    item.doSomething();  
}
```

# Organigramme : `for` sur itérables

Ce qui donne :



# Imbrication des structures de contrôle

Les structures de contrôles définissent des blocs d'instructions, entre { et } :

- possible d'utiliser d'autres structures de contrôle dans ces blocs, par **imbrication**,
- les blocs définissent les **portées** des variables, une variable ne vit que dans le bloc où elle est définie et les blocs imbriqués dedans.
- les structures de contrôle compliquent le programme : **il faut éviter l'imbrication !**